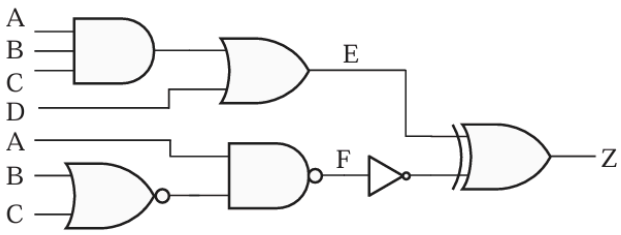
EE2016 Microprocessor and Theory Lab
VHDL Tutorial

1.

Write a VHDL description of the following combinational circuit using concurrent statements. Each gate has a 5-ns delay, excluding the inverter, which has a 2-ns delay.



2.

An M-N flip-flop responds to the falling clock edge as follows:

If $M = N =$ '0', the flip-flop changes state.
If $M =$ '0' and $N =$ '1', the flip-flop output is set to '1'.
If $M =$ '1' and $N =$ '0', the flip-flop output is set to '0'.
If $M = N =$ '1', no change of flip-flop state occurs.
The flip-flop is cleared asynchronously if $CLRn =$ '0'.

Write a complete VHDL module that implements an M-N flip-flop.

3.

In the following VHDL process A, B, C, and D are all integers that have a value of 0 at time = 10 ns. If E changes from '0' to '1' at time = 20 ns, specify the time(s) at which each signal will change and the value to which it will change. List these changes in chronological order (20, 20 + Δ, 20 + 2Δ, etc.).

```
p2: process(E)
begin
  A <= 1 after 5 ns;
  B <= A + 1;
  C <= B after 10 ns;

  D <= B after 3 ns;
  A <= A + 5 after 15 ns;
  B <= B + 7;
end process p2;
```

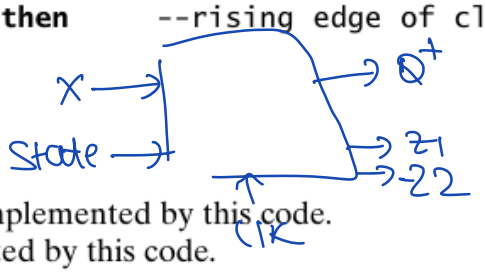
	20	20+Δ	23	25	30	35
A	0	0	0	1	1	5
B	0	7	7	7	7	7
C	0	0	0	0	0	0
D	0	0	0	0	0	0

4.

Examine the following VHDL code and complete the following exercises:

```
entity Problem
  port(X, CLK: in bit;
       Z1, Z2: out bit);
end Problem;

architecture Table of Problem is
  signal State, Nextstate: integer range 0 to 3 := 0;
begin
  process(State, X)       --Combinational Circuit
  begin
    case State is
      when 0 =>
        if X = '0' then Z1 < = '1'; Z2 <= '0'; Nextstate < = 0;
        else Z1 < = '0'; Z2 < = '0'; Nextstate < = 1; end if;
      when 1 =>
        if X = '0' then Z1 < = '0'; Z2 <= '1'; Nextstate < = 1;
        else Z1 < = '0'; Z2 < = '1'; Nextstate < = 2; end if;
      when 2 =>
        if X = '0' then Z1 < = '0'; Z2 <= '1'; Nextstate < = 2;
        else Z1 < = '0'; Z2 < = '1'; Nextstate < = 3; end if;
      when 3 =>
        if X = '0' then Z1 < = '0'; Z2 <= '0'; Nextstate < = 0;
        else Z1 < = '1'; Z2 < = '0'; Nextstate < = 1; end if;
    end case;
  end process;
  process(CLK)            --State Register
  begin
    if CLK'event and CLK = '1' then     --rising edge of clock
      State <= Nextstate;
    end if;
  end process;
end Table;
```



(a) Draw a block diagram of the circuit implemented by this code.

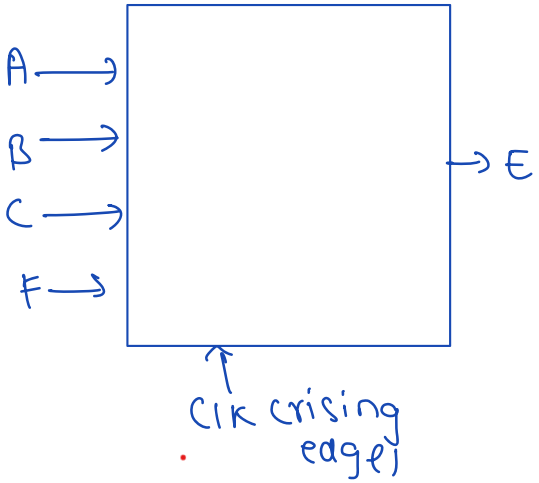
(b) Write the state table that is implemented by this code.

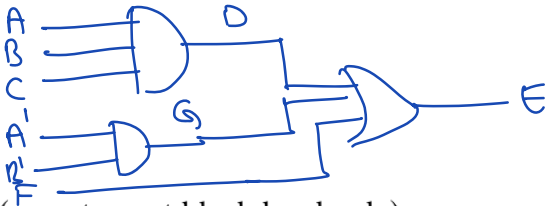
present	input(x)	Next State	Z1	Z2
0	0	0	1	0
0	1	1	0	0

Consider the following VHDL code:

```
entity Q3 is
  port(A, B, C, F, Clk: in bit;
       E: out bit);
end Q3;

architecture Qint of Q3 is
  signal D, G: bit;
begin
  process(Clk)
  begin
    if Clk'event and Clk = '1' then
      D <= A and B and C;
      G <= not A and not B;
      E <= D or G or F;
    end if;
  end process;
end Qint;
```





(a) Draw a block diagram for the circuit (no gates—at block level only).
(b) Give the circuit generated by the preceding code (at the gate level)

5.

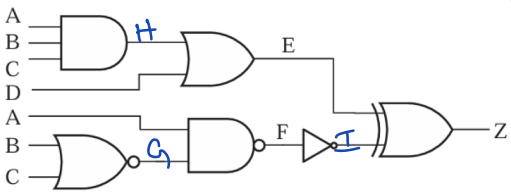
6.

A Moore sequential machine with two inputs (X_1 and X_2) and one output (Z) has the following state table:

Present State	Next State $X_1X_2 = 00$	01	10	11	Output (Z)
1	1	2	2	1	0
2	2	1	2	1	1

Write VHDL code that describes the machine at the behavioral level. Assume that state changes occur 10 ns after the falling edge of the clock, and output changes occur 10 ns after the state changes.

Write a VHDL description of the following combinational circuit using concurrent statements. Each gate has a 5-ns delay, excluding the inverter, which has a 2-ns delay.




```
entity comb is
    port(A,B,C,D: in bit;
         Z: out bit);

end comb
architecture COMB2 of comb is
    signal E,F,G,H,I: bit;
begin
    H<=(A and B and C) after 5ns;
    E<=(H or D) after 5ns;
    G<=(B nor C) after 5ns;
    F<=(G nand A) after 5ns;
    I<=(not F) after 2ns;
    Z<=E xor I after 5ns;

end comb2;
```

An M-N flip-flop responds to the falling clock edge as follows:

If $M = N =$ '0', the flip-flop changes state.
If $M =$ '0' and $N =$ '1', the flip-flop output is set to '1'.
If $M =$ '1' and $N =$ '0', the flip-flop output is set to '0'.
If $M = N =$ '1', no change of flip-flop state occurs.
The flip-flop is cleared asynchronously if $CLRn =$ '0'.

Write a complete VHDL module that implements an M-N flip-flop.

```
entity mnff is
    port(M,N,CLRn,CLK: in bit; Q: inout bit; Qn: out bit);
end mnff;
architecture MN of mnff is
  begin
  process(CLK,CLRN)
   begin
     if CLRn='0' then Q<='0';
     elsif CLK='0' and CLK'event then
```

```
        if M='0' and N='0' then Q <= not Q;
        elsif M='0' and N='1' then Q<='1';
        elsif M='1' and N='0' then Q<='0';
        elsif M='1' and N='1' then Q <= Q;
        optional endif;
    endif;
  end process;
  ON <= not Q;
end MN;
```

A Moore sequential machine with two inputs (X_1 and X_2) and one output (Z) has the following state table:

Present State	Next State X_1X_2 = 00	01	10	11	Output (Z)
1	1	2	2	1	0
2	2	1	2	1	1

Write VHDL code that describes the machine at the behavioral level. Assume that state changes occur 10 ns after the falling edge of the clock, and output changes occur 10 ns after the state changes.

```
entity Q6 is
  port (
    X1, X2, Clk: in bit;
    Z      : out bit
  );
end entity Q6;

architecture Behavioral of Q6 is
  type state_type is (S1, S2);
  signal State, Nextstate : state_type := S1;
begin

  process(State, X1, X2)
  begin
    case State is
      when S1 =>
        case (X1 & X2) is
          when "00" => Nextstate <= S1;
          when "01" => Nextstate <= S2;
          when "10" => Nextstate <= S2;
          when "11" => Nextstate <= S1;
          when others => Nextstate <= S1;
```

```
        end case;

      when S2 =>
        case (X1 & X2) is
          when "00" => Nextstate <= S2;
          when "01" => Nextstate <= S1;
          when "10" => Nextstate <= S2;
          when "11" => Nextstate <= S1;
          when others => Nextstate <= S2;
        end case;
    end case;
  end process;

  process(Clk)
  begin
    if falling_edge(Clk) then
      State <= Nextstate after 10 ns;
    end if;
  end process;

  -- Moore output: depends only on State; output changes 10 ns after the State change
  process(State)
  begin
    case State is
      when S1 => Z <= '0' after 10 ns;  -- State changed at t_s, Z changes at t_s + 10
      when S2 => Z <= '1' after 10 ns;
    end case;
  end process;

end architecture Behavioral;
```